

Matkul : Pengembangan Aplikasi Mobile
Dosen : Muhammad Aminul Akbar, S.Kom., M.T.
Modul : Mobile_4_State Hoisting, Configuration Change, LaunchedEffect-VM
Kelas : SI – D

Nama Anggota : Raihan Nasrullah Alamin
NIM : 245150401111031

Percobaan

4.5.1 State Hoisting

- 1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" di klik beberapa kali.**

A. Apa yang terjadi setiap kali tombol "Increment" di klik?

Setiap kali tombol "Increment" diklik, nilai angka yang ditampilkan pada Text akan bertambah satu. Jika nilai awal adalah 0, maka setelah satu kali klik menjadi 1, setelah dua kali klik menjadi 2, dan seterusnya. Perubahan angka terjadi secara langsung pada antarmuka aplikasi tanpa perlu menjalankan ulang aplikasi.

B. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena tombol Button memiliki event onClick yang menjalankan kode `number++`. Variabel `number` merupakan state yang dibuat menggunakan `remember { mutableStateOf(0) }`. Ketika nilai state berubah, Jetpack Compose akan melakukan recomposition pada Composable yang bergantung pada state tersebut sehingga nilai pada Text diperbarui secara otomatis.

C. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state yang berperan adalah number, yang dideklarasikan dengan:

```
var number by remember { mutableStateOf(0) }
```

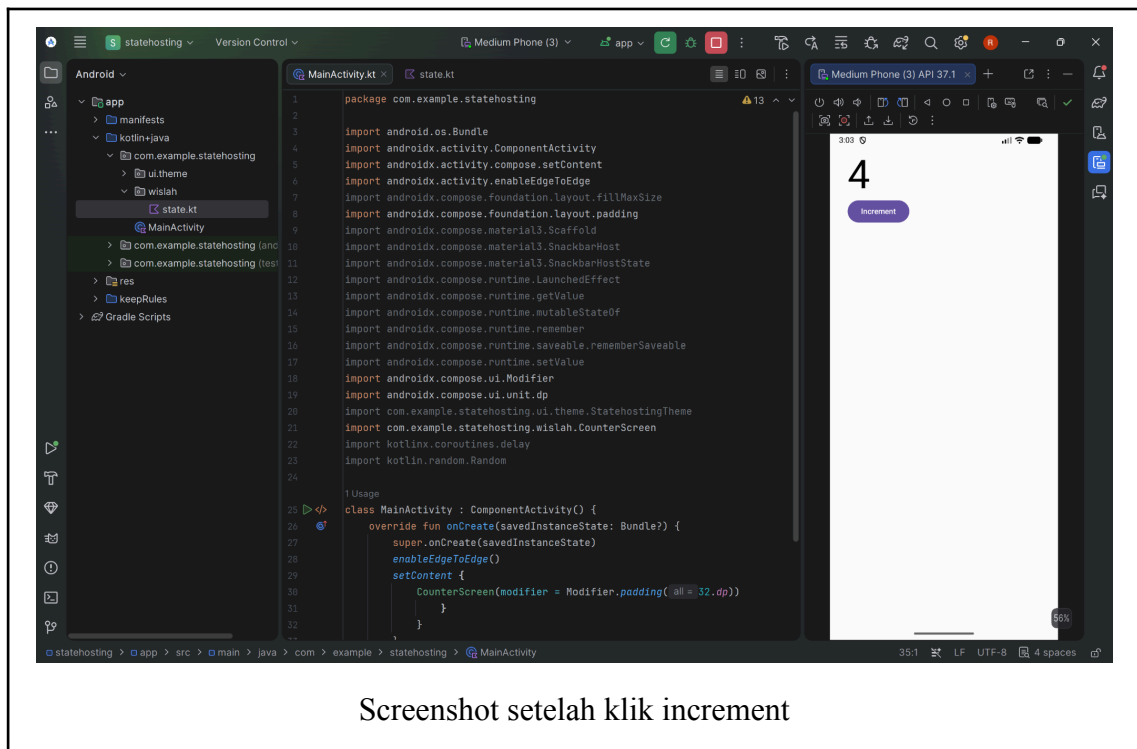
Variabel tersebut menyimpan nilai angka yang ditampilkan oleh Composable Text. Ketika number berubah akibat tombol diklik, Compose akan memperbarui tampilan Text.

D. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state yang berperan adalah label, yang dideklarasikan dengan:

```
val label by remember { mutableStateOf("Increment") }
```

Variabel label digunakan sebagai sumber teks pada Composable Text yang berada di dalam Button. Nilainya adalah "Increment" sehingga tombol menampilkan tulisan tersebut.



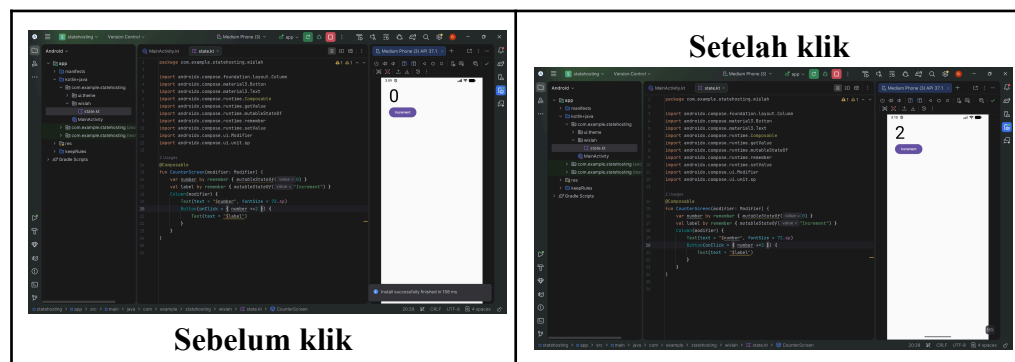
Screenshot setelah klik increment

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" di klik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

A. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Perubahan dilakukan pada Composable CounterScreen, khususnya pada bagian event onClick dari Button dan nilai awal label. Agar angka bertambah dua setiap kali tombol ditekan, kode `number++` diubah menjadi `number += 2`.

- B. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

A. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, program berjalan dengan baik dan menghasilkan perilaku yang sama seperti pada tahap nomor 4. Ketika tombol Increment diklik, nilai number tetap bertambah satu setiap klik. Perbedaannya terletak pada lokasi penyimpanan state dan event. Pada tahap sebelumnya state berada di dalam CounterScreen, sedangkan setelah dilakukan state dan event hoisting, state

number dan event `onButtonClick` dikelola oleh fungsi `onCreate()` dan dikirimkan ke `CounterScreen` sebagai parameter.

B. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment saat ini dilakukan pada lambda `onButtonClick` ketika `CounterScreen` dipanggil di dalam fungsi `onCreate()`, yaitu:

```
onButtonClick = { number++ }
```

Dengan demikian, `CounterScreen` tidak lagi menentukan bagaimana nilai number diubah. `Composable` tersebut hanya menerima event yang diberikan dari luar.

C. Di bagian kode program manakah saat ini state `Composable Text` dan `Composable Button "Increment"` dideklarasikan?

State number sekarang dideklarasikan di dalam fungsi `onCreate()`:

```
var number by remember { mutableStateOf(0) }
```

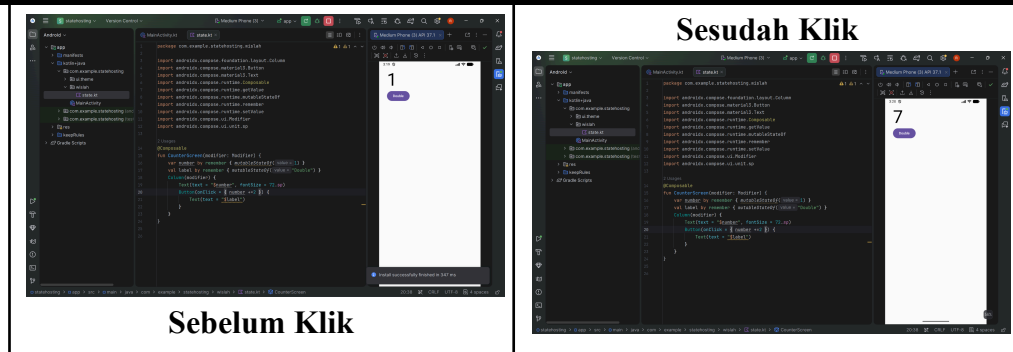
Sementara label "Increment" diberikan sebagai parameter ketika memanggil `CounterScreen`:

```
label = "Increment"
```

`CounterScreen` kemudian menggunakan kedua nilai tersebut untuk menampilkan `Text` dan label pada `Button`.

6. Jalankan program,

A. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.



B. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Pada tahap nomor 7, proses increment dilakukan pada bagian pemanggilan CounterScreen di dalam fungsi onCreate(), tepatnya pada lambda `onButtonClick = { number++ }`. Pada tahap nomor 8, bagian tersebut diubah menjadi `onButtonClick = { number *= 2 }`. Selain itu, perubahan nilai awal number dan label tombol juga dilakukan pada bagian pemanggilan CounterScreen di dalam onCreate(). Dengan demikian, perubahan logika aplikasi pada kedua tahap tersebut dilakukan di luar Composable CounterScreen.

C. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak terdapat perubahan pada fungsi Composable CounterScreen antara tahap nomor 7 dan tahap nomor 8. Struktur CounterScreen tetap sama karena Composable tersebut hanya menerima number, label, dan onButtonClick sebagai parameter. Perubahan yang dilakukan pada tahap nomor 8 hanya terdapat pada nilai state awal, label tombol, dan logika event yang diberikan ketika CounterScreen dipanggil. Hal ini menunjukkan bahwa CounterScreen tidak perlu diubah ketika terjadi perubahan logika pada aplikasi.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

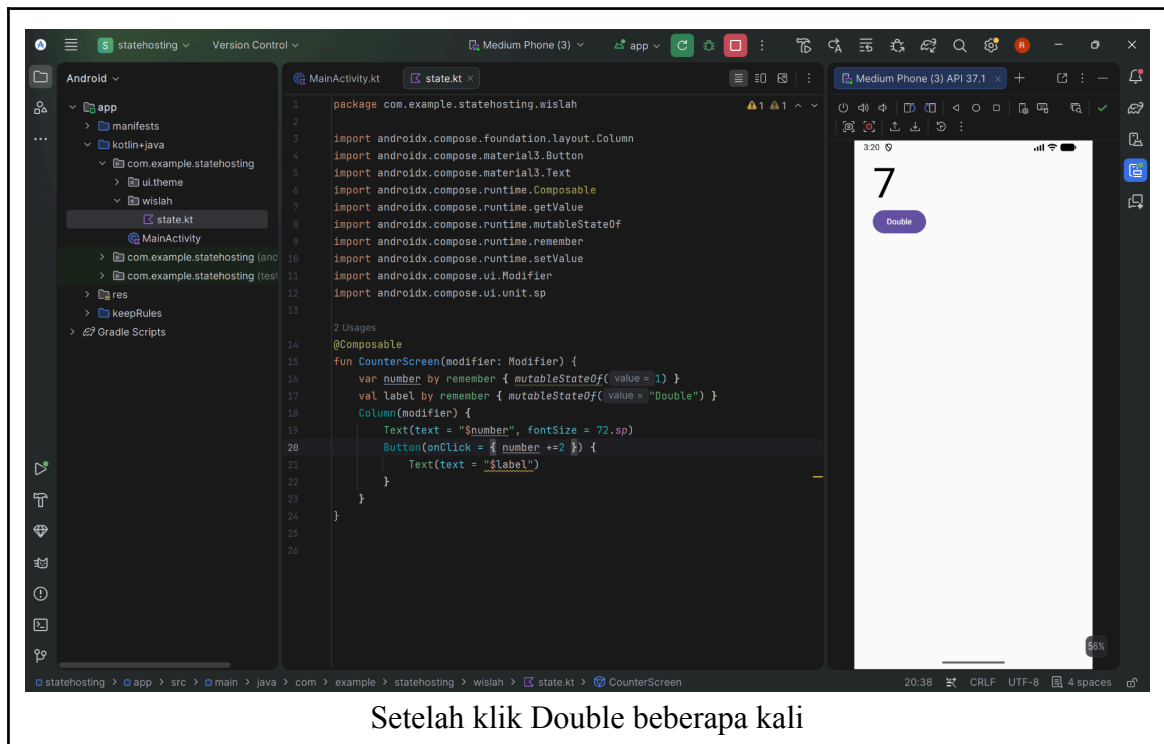
State hoisting memberikan manfaat dalam mempermudah pengelolaan dan perubahan logika aplikasi karena state dan event tidak lagi dikelola langsung di dalam Composable CounterScreen. Berdasarkan percobaan pada tahap nomor 7 dan nomor 8, perubahan kebutuhan dari menambahkan nilai menggunakan `number++` menjadi menggandakan nilai menggunakan `number *= 2` dapat dilakukan hanya pada fungsi `onCreate()` tanpa perlu mengubah struktur CounterScreen.

Dengan adanya state hoisting, CounterScreen menjadi lebih sederhana, fleksibel, dan dapat digunakan kembali karena hanya bertanggung jawab terhadap tampilan antarmuka berdasarkan data dan event yang diterima. Sementara itu, pengelolaan state dan logika perubahan nilai dilakukan oleh bagian yang lebih tinggi. Oleh karena itu, ketika terjadi perubahan kebutuhan fungsional yang berkaitan dengan antarmuka pengguna, pengembang dapat mengubah logika state tanpa harus melakukan perubahan langsung pada Composable yang menangani tampilan.

4.5.2 Perubahan Konfigurasi

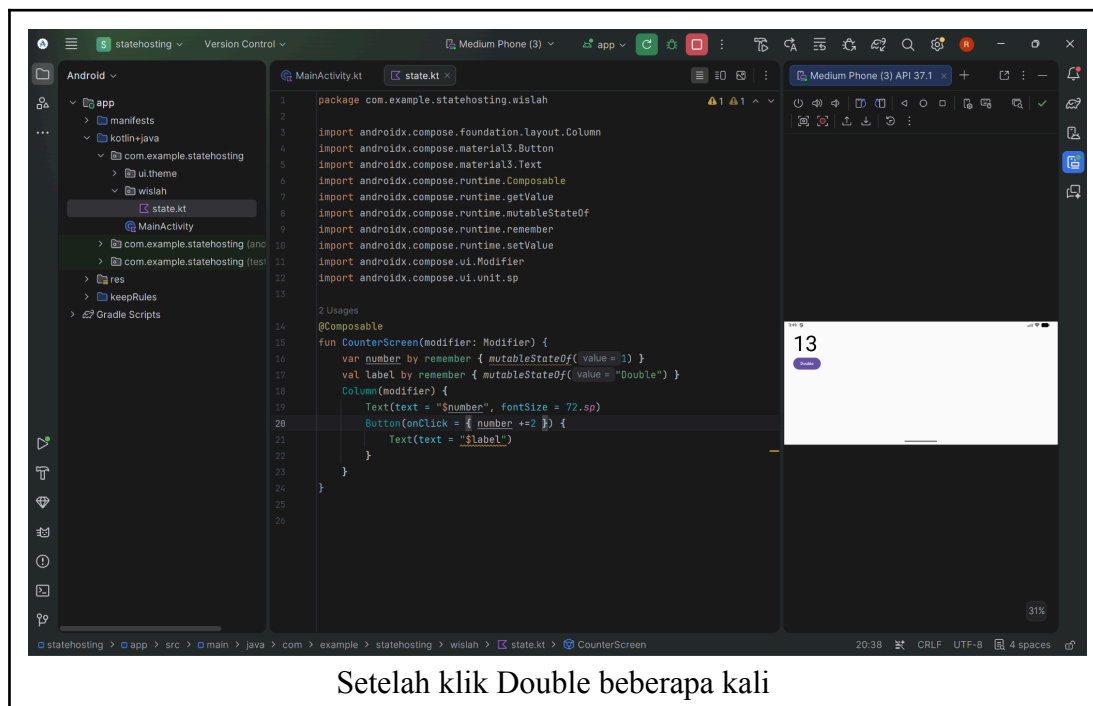
1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

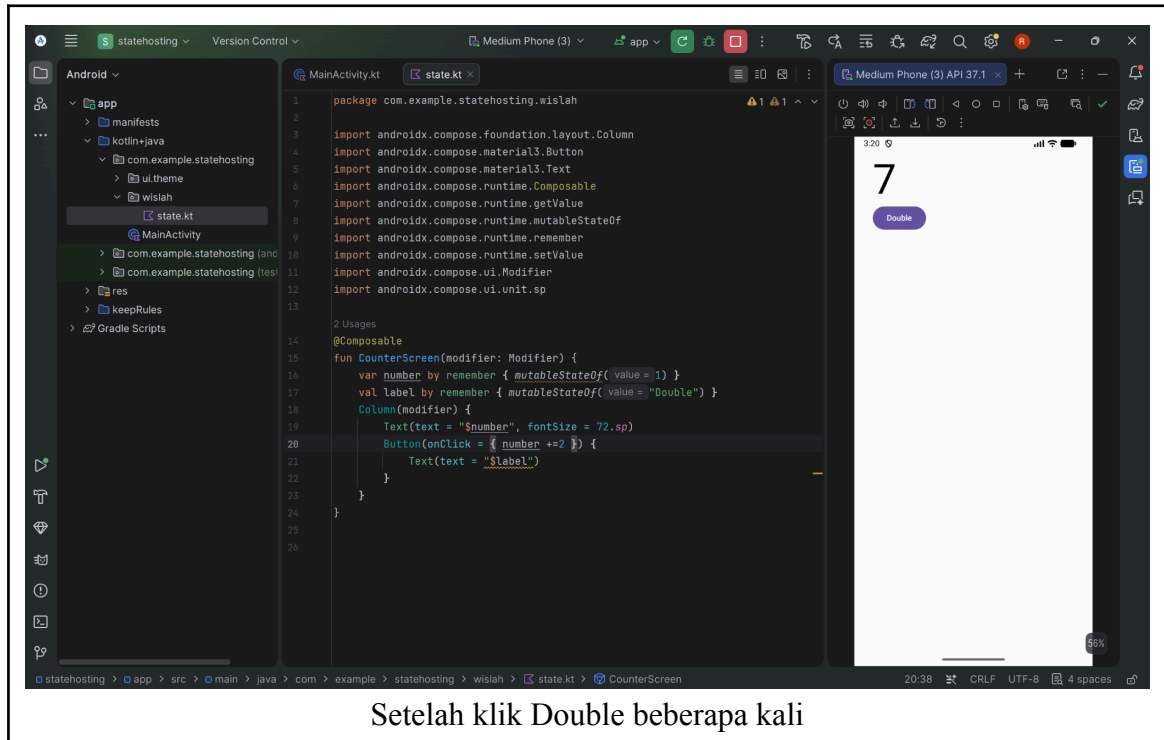
a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

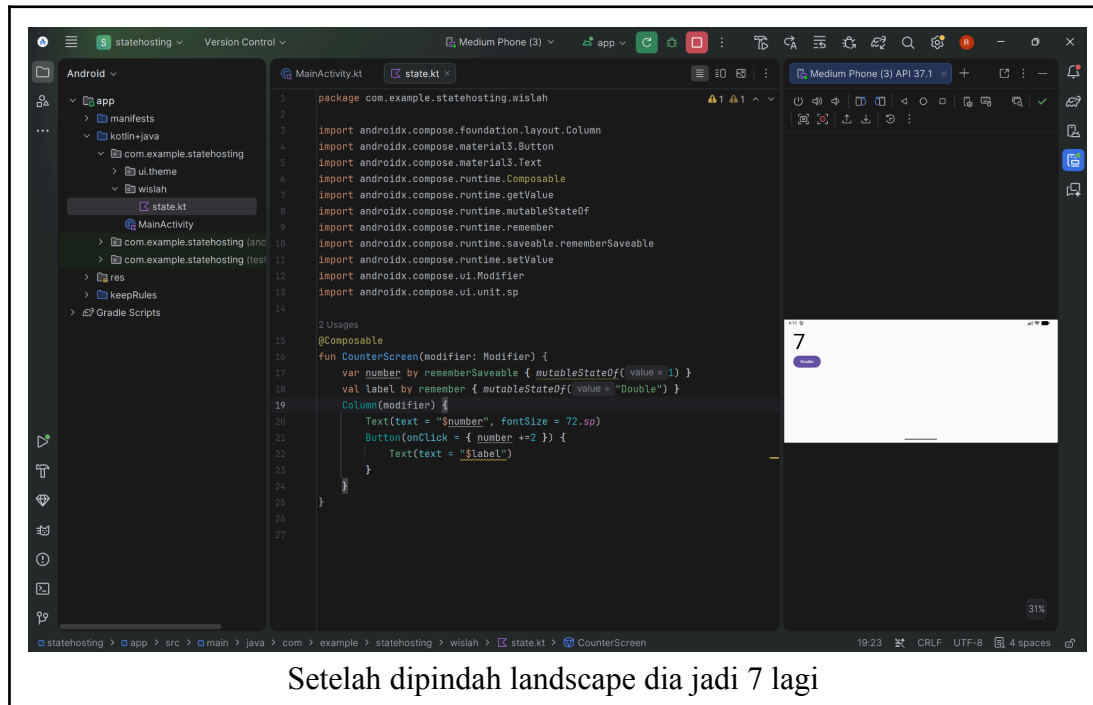
Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan setelah perangkat diputar dari portrait ke landscape adalah 7 atau sesuai dengan angka terakhir yang diperoleh sebelum perubahan konfigurasi. Hal tersebut terjadi karena variabel number sekarang menggunakan rememberSaveable. Konstruk tersebut dapat menyimpan nilai state dan memulihkannya kembali ketika terjadi perubahan konfigurasi seperti rotasi layar. Oleh karena itu, meskipun Activity mengalami pembuatan ulang akibat perubahan konfigurasi, nilai number yang sebelumnya telah diperoleh tidak kembali ke nilai awal 1.

c. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?

Konstruk rememberSaveable berfungsi untuk mempertahankan dan memulihkan nilai state ketika terjadi perubahan konfigurasi atau ketika Activity perlu dibuat kembali. Perbedaannya dengan remember adalah remember hanya mempertahankan state selama recomposition dalam lifecycle komposisi yang sama, sedangkan rememberSaveable dapat

menyimpan state ke mekanisme penyimpanan instance sehingga state dapat dipulihkan setelah configuration change, seperti perubahan orientasi layar. Dalam percobaan ini, penggunaan `rememberSaveable` menyebabkan nilai number tetap sama setelah perangkat diputar dari portrait ke landscape.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Salah satu contoh penggunaan `rememberSaveable` yang tepat adalah pada form pendaftaran atau pengisian data pengguna dalam aplikasi Android. Misalnya, pengguna sedang mengisi nama, alamat, atau nomor telepon pada beberapa kolom formulir kemudian perangkat diputar dari portrait menjadi landscape. Jika nilai input disimpan menggunakan `rememberSaveable`, data yang telah dimasukkan dapat dipertahankan dan ditampilkan kembali setelah terjadi perubahan konfigurasi. Dengan demikian, pengguna tidak perlu mengisi ulang seluruh data yang sebelumnya sudah dimasukkan. Penerapan `rememberSaveable` pada kasus tersebut memberikan manfaat berupa meningkatkan pengalaman pengguna, mencegah kehilangan data sementara, serta mempertahankan kondisi antarmuka ketika terjadi perubahan konfigurasi perangkat.

4.5.3 LaunchedEffect

Pra Percobaan 1

1. Jalankan program,

A. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Pada saat pertama kali program dijalankan, layar menampilkan nilai Number 1. Selain itu, muncul `SnackBar` dengan pesan "Number 1 is shown!".

B. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Pesan tersebut ditampilkan karena `LaunchedEffect(number)` akan dijalankan ketika `composable` pertama kali masuk ke dalam komposisi. Pada kondisi awal, variabel `number` memiliki nilai 1, sehingga `LaunchedEffect` langsung menjalankan kode di dalamnya meskipun belum ada interaksi dari pengguna.

C. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Pesan `SnackBar` dipicu oleh nilai `number` yang digunakan sebagai `key` pada `LaunchedEffect`. Ketika `number` pertama kali memiliki nilai 1, efek dijalankan dan `SnackBar` ditampilkan. Kode yang memicunya adalah:

```
LaunchedEffect(number) {  
  
    snackbarHostState.showSnackBar("Number $number is shown!")  
  
}
```

2. Klik button "Double" satu kali,

A. Apa yang terjadi?

Setelah button "Double" diklik satu kali, nilai `Number` berubah dari 1 menjadi 2. Setelah itu, muncul `SnackBar` dengan pesan "Number 2 is shown!"

B. Mengapa hal tersebut terjadi? Jelaskan!

Hal tersebut terjadi karena button "Double" menyebabkan nilai `number` berubah dari 1 menjadi 2. Karena `number` digunakan sebagai `key` pada `LaunchedEffect(number)`, perubahan nilai tersebut menyebabkan `LaunchedEffect` dijalankan kembali. Selanjutnya, `snackbarHostState.showSnackBar()` menampilkan pesan "Number 2 is shown!". Jadi, setiap kali nilai `number` berubah, `LaunchedEffect` akan menjalankan kembali efek yang ada di dalamnya.

Pra Percobaan 2

1. Jalankan program,

A. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Pada saat pertama kali program dijalankan, layar menampilkan nilai awal Number 1. Snackbar juga akan menampilkan pesan "Number 1 is shown!" karena `LaunchedEffect(number)` dijalankan ketika aplikasi pertama kali masuk ke dalam komposisi.

B. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah menunggu selama dua detik, nilai number berubah dari 1 menjadi angka acak antara 100 sampai 999. Snackbar kemudian menampilkan pesan sesuai dengan angka baru tersebut, misalnya "Number 537 is shown!". Angka yang muncul dapat berbeda setiap kali program dijalankan karena menggunakan `Random.nextInt(100, 1000)`.

C. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan angka dipicu oleh `LaunchedEffect(Unit)` yang menjalankan `Repo.getData()`. Fungsi tersebut memiliki `delay(2000)` sehingga setelah dua detik akan mengembalikan angka acak dan memasukkannya ke dalam variabel `number`.

Bagian kode yang memicu perubahan angka adalah:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

Setelah number berubah, `LaunchedEffect(number)` yang sudah dibuat pada percobaan sebelumnya akan dijalankan kembali karena nilai number sebagai key berubah. Bagian kode yang menampilkan `SnackBar` adalah:

```
LaunchedEffect(number) {  
  
    snackbarHostState.showSnackBar("Number $number is  
    shown!")  
  
}
```

Jadi, `LaunchedEffect(Unit)` mengubah nilai number, sedangkan perubahan number memicu `LaunchedEffect(number)` untuk menampilkan `SnackBar`.

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

A. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

Perbedaannya terletak pada key yang digunakan untuk menentukan kapan `LaunchedEffect` dijalankan kembali. `LaunchedEffect(state)` menggunakan nilai state sebagai key, sehingga efek akan dijalankan ketika composable pertama kali masuk ke dalam komposisi dan akan dijalankan kembali apabila nilai state berubah. Sementara itu, `LaunchedEffect(Unit)` menggunakan nilai tetap `Unit` sebagai key, sehingga efek hanya dijalankan ketika composable pertama kali masuk ke dalam komposisi dan tidak dijalankan kembali hanya karena terjadi perubahan state lain.

B. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

`LaunchedEffect(state)` dijalankan ketika composable pertama kali masuk ke dalam komposisi dan akan dijalankan kembali setiap kali nilai state yang menjadi key berubah. Dalam percobaan ini, ketika number berubah dari 1 menjadi angka acak, `LaunchedEffect(number)` dijalankan kembali dan menampilkan `SnackBar`.

LaunchedEffect(Unit) dijalankan ketika composable pertama kali masuk ke dalam komposisi. Karena Unit merupakan key yang tetap, efek tersebut tidak dijalankan kembali akibat perubahan state. Dalam percobaan ini, efek tersebut digunakan untuk memanggil Repo.getData() satu kali saat awal aplikasi dijalankan, dengan waktu tunggu dua detik sebelum mendapatkan angka acak.